Name - Viraj Shyam Bhiwandkar
Batch - T11
Roll No 2401014
Branch - AI&DS

Experiment 3

Aim - To deploy multiservice applications using docker compose
LO : LO4

Docker Compose is a tool that allows you to define, configure, and manage multi-container Docker applications.

Instead of typing out multiple long, complex `docker run` commands in your terminal to start each individual container, you write a single configuration file that describes exactly how all your containers should be set up and how they should interact with one another.

How It Works

Docker Compose uses a YAML file (typically named `docker-compose.yml`) to outline your application's architecture. Once the file is written, you use a single command line instruction to start, stop, or rebuild all the services defined in that file simultaneously.

docker-compose.yml

```yaml
# Specify the version of the Docker Compose file format
version: '3.8'

services:
  # Service 1: The Relational Database
  db:
    # Use the official MySQL 8.0 image from Docker Hub
    image: mysql:8.0
    # Assign a custom name to the container for easier management
    container_name: lab_mysql_db
    # Set essential environment variables for MySQL initialization
    environment:
      # Define the root user's password
      MYSQL_ROOT_PASSWORD: rootpassword
      # Automatically create a database named 'lab_database' on startup
      MYSQL_DATABASE: lab_database
    ports:
      # Map port 3307 on your local machine to port 3306 inside the
container
```

```
      - "3307:3306"
    volumes:
      # Mount the persistent volume 'db_data' to store database files
securely
      - db_data:/var/lib/mysql

  # Service 2: The Web Interface (Adminer)
  adminer:
    # Use the official Adminer image, a lightweight database management
tool
    image: adminer
    # Assign a custom name to the container
    container_name: lab_web_admin
    # Ensure the container automatically restarts if it crashes or the
system reboots
    restart: always
    ports:
      # Map port 8081 on your local machine to port 8080 inside the
container
      - "8081:8080"
    depends_on:
      # Tell Docker to start the 'db' service before starting Adminer
      - db

  # Service 3: The Backend API (Newly Added)
  api_backend:
    # Use the official Node.js version 18 image as the environment
    image: node:18
    # Assign a custom name to your backend container
    container_name: lab_api_backend
    # Set the working directory inside the container to /app
    working_dir: /app
    # Map your local project folder (using current directory ./) to /app
in the container
    volumes:
      - ./:/app
    # Command to install dependencies and start your server when the
container runs
    command: sh -c "npm install && npm start"
    ports:
```

```
      # Map port 5000 on your local machine to port 5000 inside the
container
      - "5000:5000"
    depends_on:
      # Ensure the database is up and running before starting the API
server
      - db

# Define the persistent volume for the database to prevent data loss
volumes:
  # Create a named volume managed by Docker for the MySQL data
  db_data:
```

package.json

```
{
  "name": "lab_api_backend",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mysql2": "^3.6.0"
  }
}
```

index.js

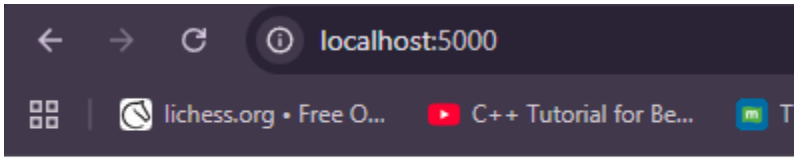
```
// Import the Express framework to create your web server
const express = require('express');

// Initialize the Express application instance
const app = express();

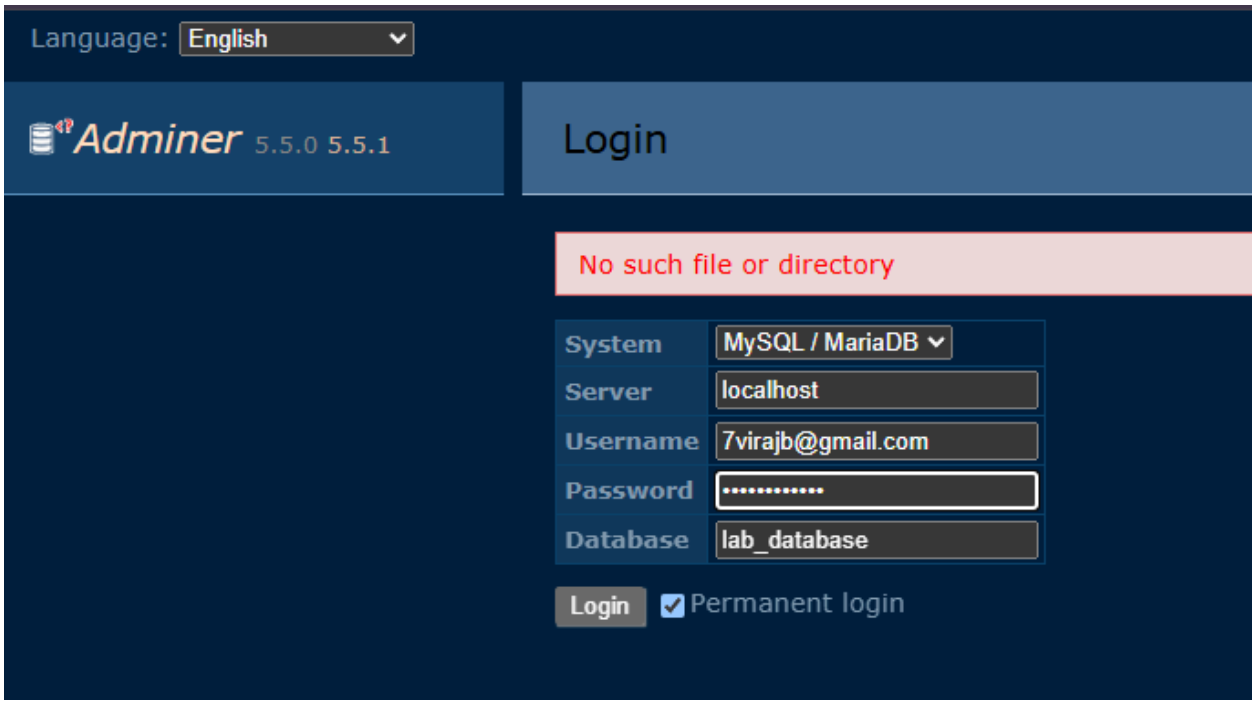
// Define the port number matching the one in docker-compose.yml
const port = 5000;
```

```
// Create a basic GET route to test if the server is working
app.get('/', (req, res) => {
  // Send a simple text response when someone visits the root URL
  res.send('Backend API is successfully running!');
});

// Start the server and instruct it to listen on the defined port
app.listen(port, () => {
  // Log a confirmation message to the terminal once the server is up
  console.log(`Server is running on http://localhost:${port}`);
});
```

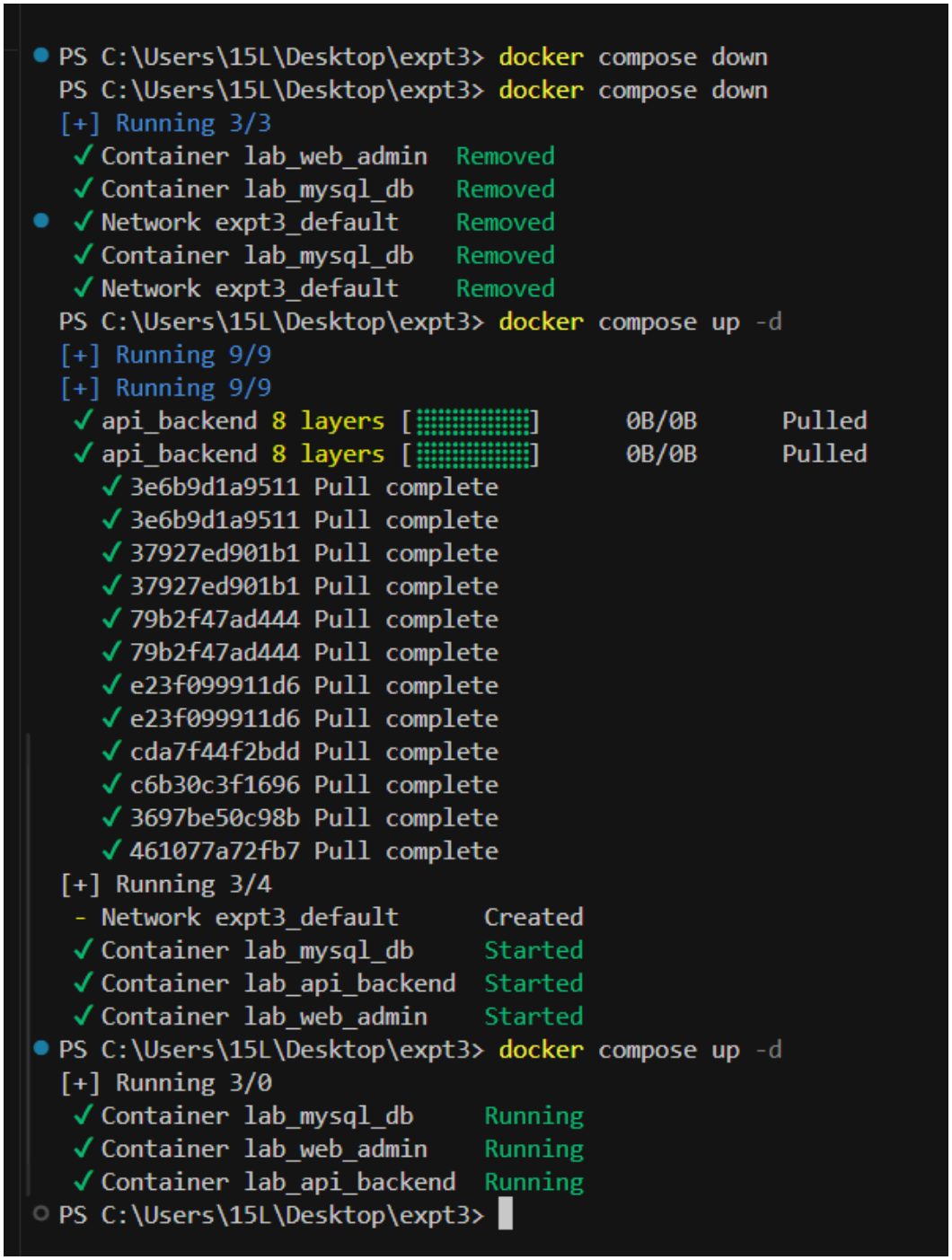


Backend API is successfully running!

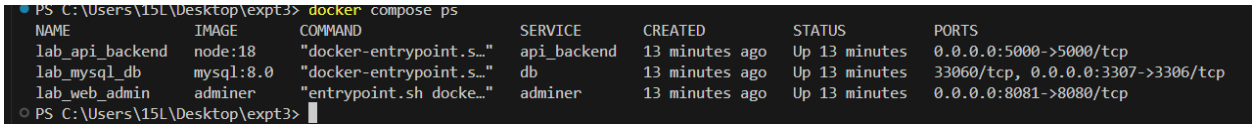


docker compose up -d -

The command **`docker compose up -d`** is the standard way to start all the services defined in your `docker-compose.yml` file and run them in the background.



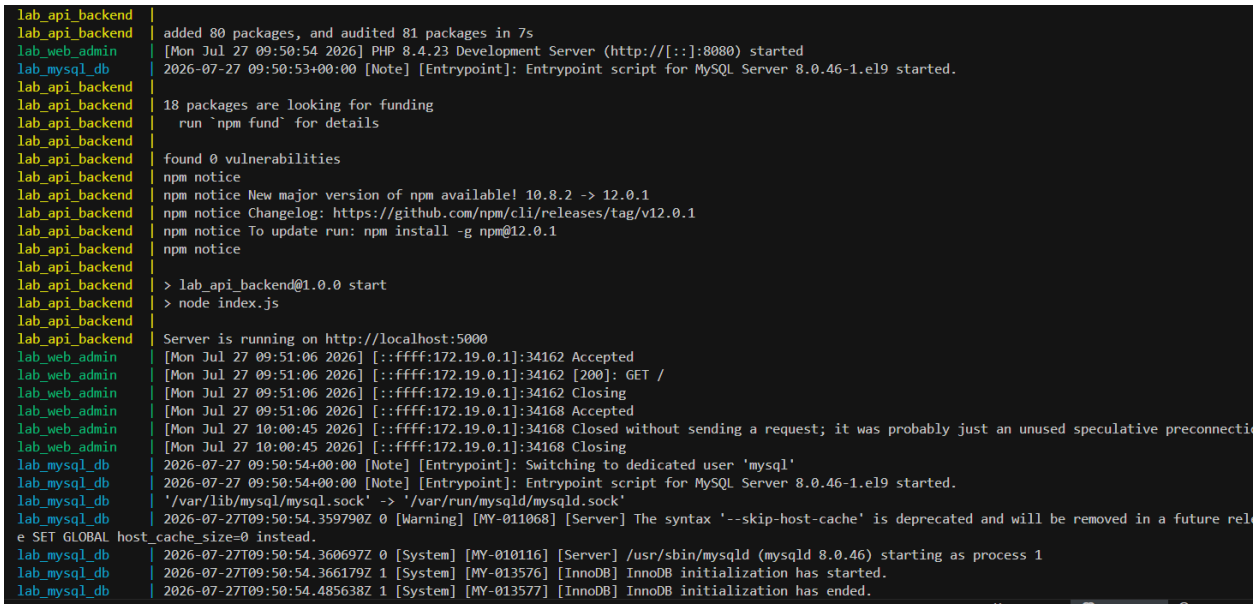
docker compose ps - The command **`docker compose ps`** is used to list all the containers that are currently running (or stopped) as part of your specific `docker-compose.yml` project, along with their current status.



```
PS C:\Users\15L\Desktop\expt3> docker compose ps
NAME              IMAGE        COMMAND                  SERVICE       CREATED          STATUS          PORTS
lab_api_backend   node:18      "docker-entrypoint.s…"   api_backend   13 minutes ago   Up 13 minutes   0.0.0.0:5000->5000/tcp
lab_mysql_db      mysql:8.0    "docker-entrypoint.s…"   db            13 minutes ago   Up 13 minutes   33060/tcp, 0.0.0.0:3307->3306/tcp
lab_web_admin     adminer      "entrypoint.sh docke…"   adminer       13 minutes ago   Up 13 minutes   0.0.0.0:8081->8080/tcp
PS C:\Users\15L\Desktop\expt3>
```

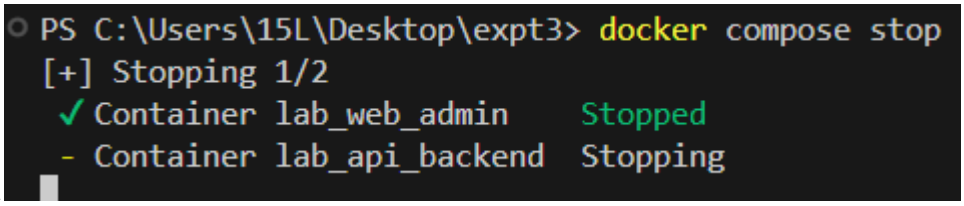
docker compose logs -
The command **`docker compose logs`** is used to view the console output (logs) generated by all the containers running in your current `docker-compose.yml` stack.

Whenever an application inside a container prints messages, warnings, or errors to its terminal (standard output or standard error), Docker captures that information. This command is your primary tool for debugging and seeing what is happening inside your services in real-time.



```
lab_api_backend  |
lab_api_backend  | added 80 packages, and audited 81 packages in 7s
lab_web_admin    | [Mon Jul 27 09:50:54 2026] PHP 8.4.23 Development Server (http://[::]:8080) started
lab_mysql_db     | 2026-07-27 09:50:53+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
lab_api_backend  |
lab_api_backend  | 18 packages are looking for funding
lab_api_backend  |   run `npm fund` for details
lab_api_backend  |
lab_api_backend  | found 0 vulnerabilities
lab_api_backend  | npm notice
lab_api_backend  | npm notice New major version of npm available! 10.8.2 -> 12.0.1
lab_api_backend  | npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.0.1
lab_api_backend  | npm notice To update run: npm install -g npm@12.0.1
lab_api_backend  | npm notice
lab_api_backend  |
lab_api_backend  | > lab_api_backend@1.0.0 start
lab_api_backend  | > node index.js
lab_api_backend  |
lab_api_backend  | Server is running on http://localhost:5000
lab_web_admin    | [Mon Jul 27 09:51:06 2026] [::ffff:172.19.0.1]:34162 Accepted
lab_web_admin    | [Mon Jul 27 09:51:06 2026] [::ffff:172.19.0.1]:34162 [200]: GET /
lab_web_admin    | [Mon Jul 27 09:51:06 2026] [::ffff:172.19.0.1]:34162 Closing
lab_web_admin    | [Mon Jul 27 09:51:06 2026] [::ffff:172.19.0.1]:34168 Accepted
lab_web_admin    | [Mon Jul 27 10:00:45 2026] [::ffff:172.19.0.1]:34168 Closed without sending a request; it was probably just an unused speculative preconnecti
lab_web_admin    | [Mon Jul 27 10:00:45 2026] [::ffff:172.19.0.1]:34168 Closing
lab_mysql_db     | 2026-07-27 09:50:54+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
lab_mysql_db     | 2026-07-27 09:50:54+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
lab_mysql_db     | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
lab_mysql_db     | 2026-07-27T09:50:54.359790Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future rel
e SET GLOBAL host_cache_size=0 instead.
lab_mysql_db     | 2026-07-27T09:50:54.360697Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.46) starting as process 1
lab_mysql_db     | 2026-07-27T09:50:54.366179Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
lab_mysql_db     | 2026-07-27T09:50:54.485638Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
```

docker compose stop - The command **`docker compose stop`** is used to halt running containers associated with your `docker-compose.yml` file without destroying them.



```
PS C:\Users\15L\Desktop\expt3> docker compose stop
[+] Stopping 1/2
 ✓ Container lab_web_admin    Stopped
 - Container lab_api_backend  Stopping
```

\

docker compose down - The command **`docker compose down`** is the teardown command for your multi-container environment. It stops running containers and then completely removes them, along with any default networks that were created when you ran `docker compose up`.

```
PS C:\Users\15L\Desktop\expt3> docker compose down
 [+] Running 4/4
  ✓ Container lab_api_backend  Removed
  ✓ Container lab_web_admin    Removed
  ✓ Container lab_mysql_db     Removed
  ✓ Network expt3_default      Removed
PS C:\Users\15L\Desktop\expt3>
```

docker compose images - The command **`docker compose images`** is used to list the Docker images that were downloaded or built to run the containers in your specific `docker-compose.yml` environment.

While `docker compose ps` shows you the *running containers*, `docker compose images` shows you the *blueprint files* (images) that were used to create those containers

```
PS C:\Users\15L\Desktop\expt3> docker compose images
CONTAINER           REPOSITORY          TAG                 IMAGE ID            SIZE
```

docker compose config - The command **`docker compose config`** is used to validate and view the final, parsed version of your `docker-compose.yml` file.

```
PS C:\Users\15L\Desktop\expt3> docker compose config
name: expt3
services:
  adminer:
    container_name: lab_web_admin
    depends_on:
      db:
        condition: service_started
        required: true
    image: adminer
    networks:
      default: null
    ports:
      - mode: ingress
        target: 8080
        published: "8081"
        protocol: tcp
    restart: always
  api_backend:
    command:
      - sh
      - -c
      - npm install && npm start
    container_name: lab_api_backend
    depends_on:
      db:
        condition: service_started
        required: true
    image: node:18
    networks:
```